

DIGITAL LOGIC DESIGN

(CE_118)

CHAPTER 3:

Register Transfer Specification & Design (part_2)

Outline

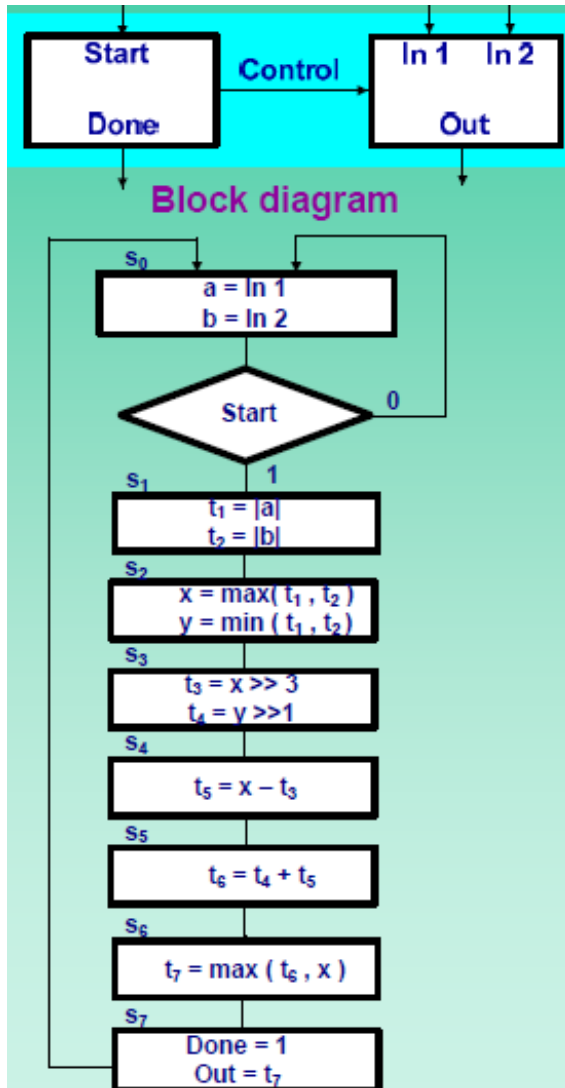
- **Register Transfer (RT) design:**
 - FSMD model
 - RT specification with
 - Static-action tables
 - ASM charts
- **Procedure for synthesis from RT specification**
- **Design Optimization through**
 - Register sharing
 - Functional unit sharing
 - Bus sharing
 - Unit chaining and Multiclocking
- **Design Pipelining**
 - Unit pipelining
 - Control pipelining
 - Datapath pipelining
- **Scheduling of flowcharts**

Register-transfer synthesis (synthesis from ASM chart)

- Register sharing
- Functional unit sharing
- Bus sharing
(datapath connectivity)

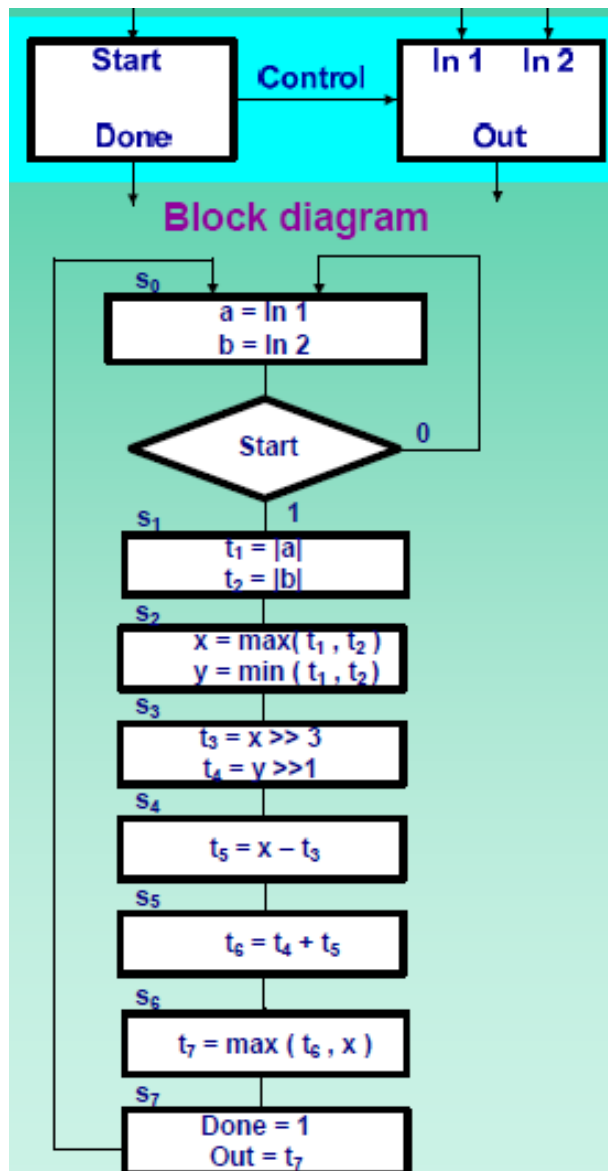
$$\sqrt{a^2+b^2} \approx \max((0,875x+0,5y),x)$$

where $x = \max(|a|,|b|)$
 $y = \min(|a|,|b|)$



ASM Chart of Square-root approximation

Resource usage in square-root approximation



	s ₁	s ₂	s ₃	s ₄	s ₅	s ₆	s ₇
a	X						
b	X						
t ₁		X					
t ₂		X					
x			X	X	X	X	
y			X				
t ₃				X			
t ₄				X	X		
t ₅					X		
t ₆						X	
t ₇							X
No. of live variables	2	2	2	3	3	2	1

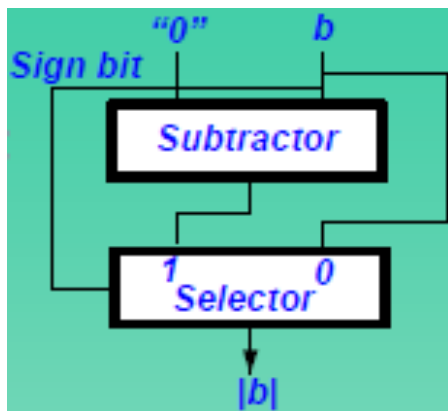
Variable usage (register/memory sharing)

	s ₁	s ₂	s ₃	s ₄	s ₅	s ₆	s ₇	Max. no. of units
abs	2							2
min		1						1
max		1				1		1
>>			2					2
-				1				1
+					1			1
No. of operations	2	2	2	1	1	1		

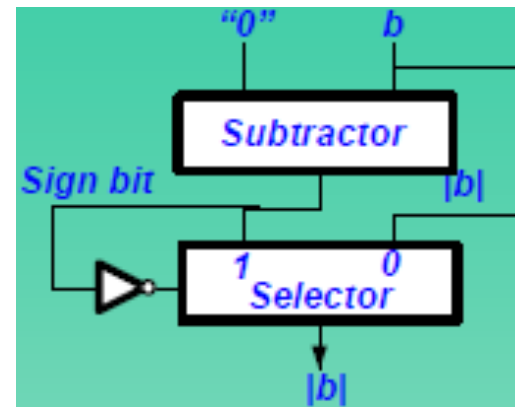
Operation usage (functional-unit sharing)

ASM Chart of Square-root approximation

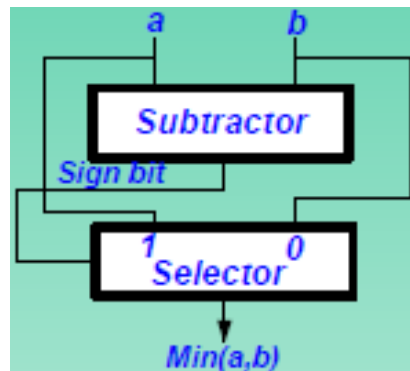
Simple library components



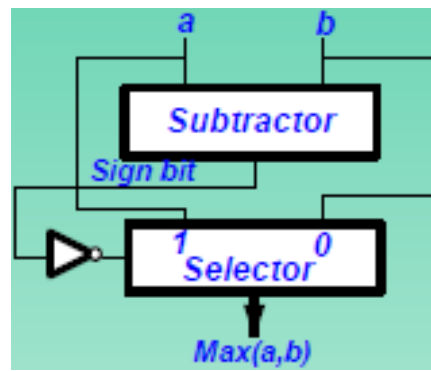
a) Absolute value unit (version 1)



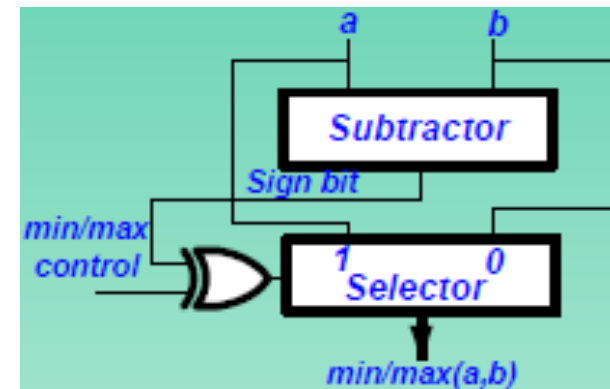
b) Absolute value unit (version 2)



(c) Min unit

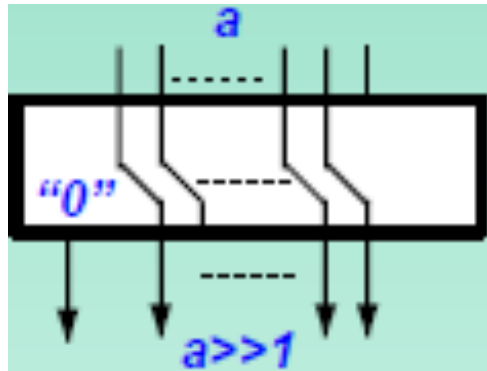


(d) Max unit

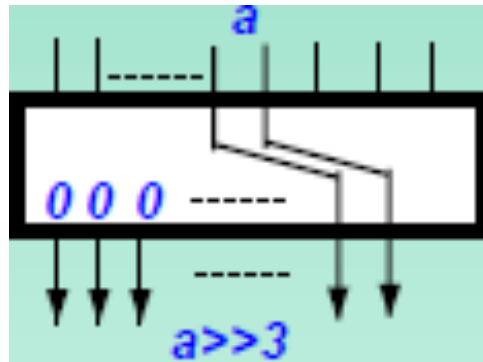


(e) Min/Max unit

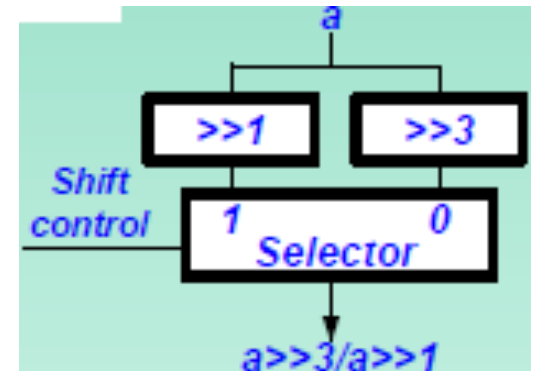
Simple library components (cont.)



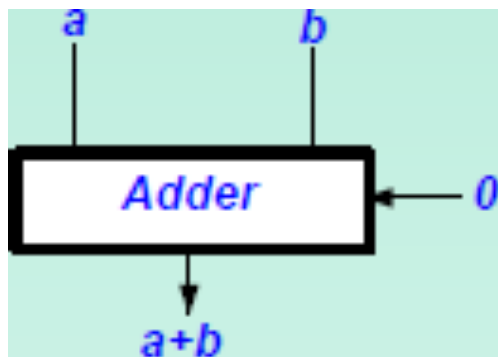
(f) 1-bit right shifter



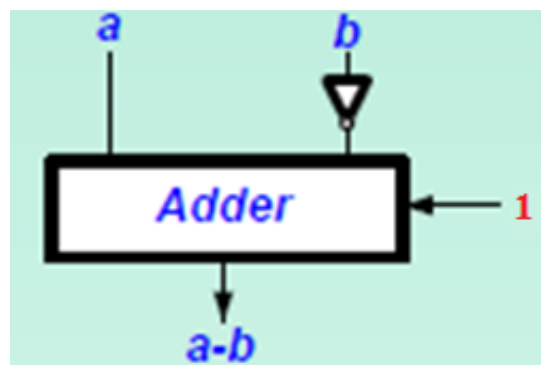
(g) 3-bit right shifter



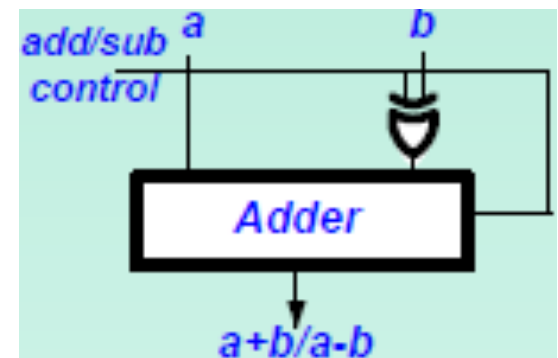
(h) 1-bit/3-bit right shifter



(i) Adder

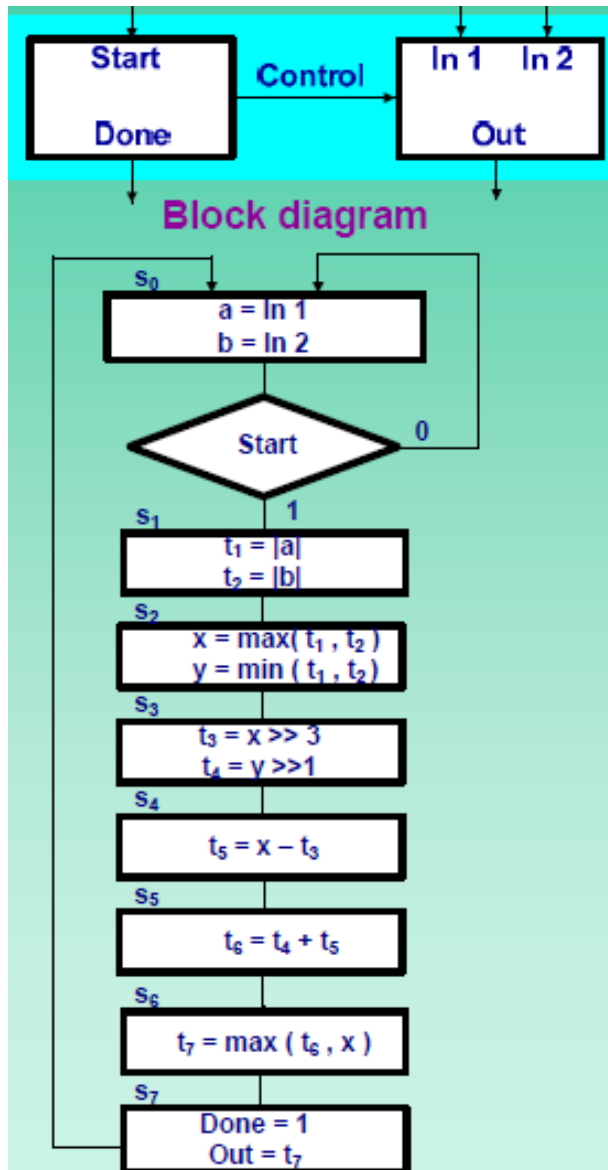


(j) Subtractor



(k) Adder/Subtractor

Connectivity requirements



	<i>a</i>	<i>b</i>	<i>t</i> ₁	<i>t</i> ₂	<i>x</i>	<i>y</i>	<i>t</i> ₃	<i>t</i> ₄	<i>t</i> ₅	<i>t</i> ₆	<i>t</i> ₇
<i>abs1</i>	<i>I</i>		<i>O</i>								
<i>abs2</i>		<i>I</i>		<i>O</i>							
<i>min</i>			<i>I</i>	<i>I</i>	<i>O</i>						
<i>max</i>			<i>I</i>	<i>I</i>	<i>I</i>	<i>O</i>				<i>I</i>	<i>O</i>
>>3					<i>I</i>		<i>O</i>				
>>1						<i>I</i>		<i>O</i>			
-					<i>I</i>		<i>I</i>		<i>O</i>		
+								<i>I</i>	<i>I</i>	<i>O</i>	

Connectivity table (bus sharing)

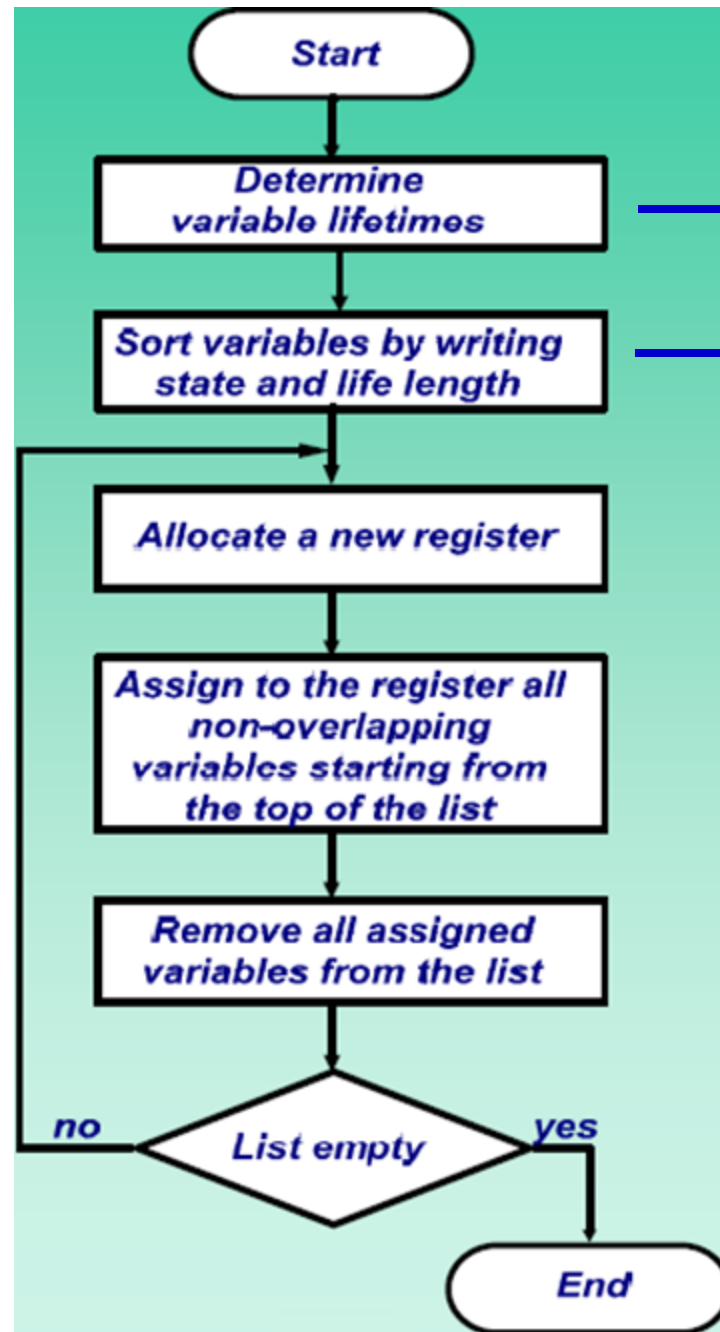
Register sharing (Variable merging)

- Grouping of variables with **nonoverlapping lifetimes** in a register

Variables that are not used (write or read) in the same state are called **nonoverlapping lifetime variables**.

- **Lifetime of a variable**: set of states in which variable is alive. It is from the state following the state variable is written (write state) to the last state that variable is used (read state), include all states between the first and last state.
- Grouping reduces number of registers needed in the design → Reduce area and cost
- Two algorithms:
 - *left-edge*
 - *graph-partitioning*

Left-edge algorithm



→ Create variables usage table

→ Sort list of variables

Register sharing with left-edge algorithm

	s_1	s_2	s_3	s_4	s_5	s_6	s_7
a	×						
b	×						
t_1		×					
t_2		×					
x			×	×	×	×	
y			×				
t_3				×			
t_4				×	×		
t_5					×		
t_6						×	
t_7							×
Number of live variables	2	2	2	3	3	2	1

Variable usage table

Creating priority list

	s_0	s_1	s_2	s_3	s_4	s_5	s_6	s_7
a		×						
b		×						
t_1			×					
t_2			×					
x				×	×	×	×	
y				×				
t_4					×	×		
t_3					×			
t_5						×		
t_6							×	
t_7								×

Sorted list of variables

Priority order for 2 variables have the **same write state**:

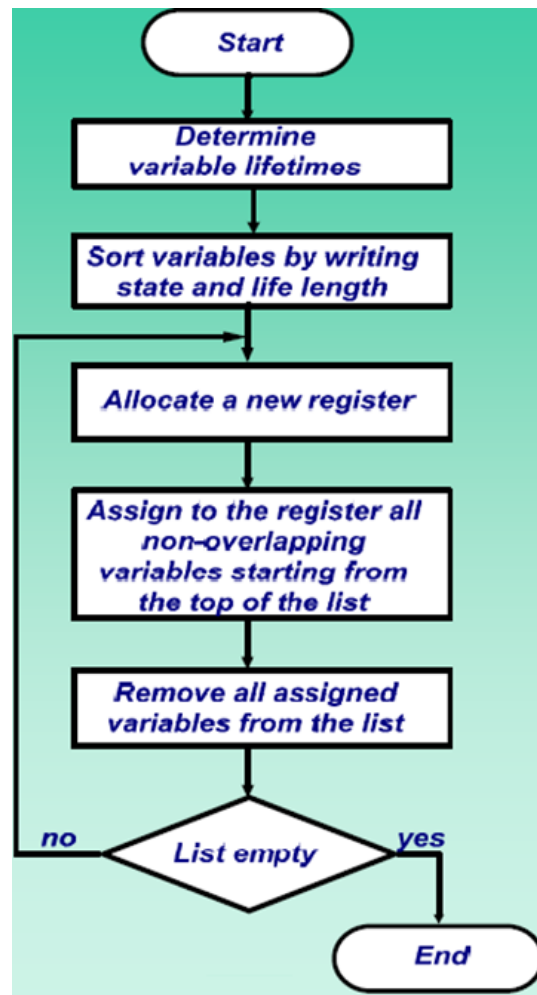
1. The variable with the longer lifetime
2. The priority will be assigned at random

Register sharing with left-edge algorithm

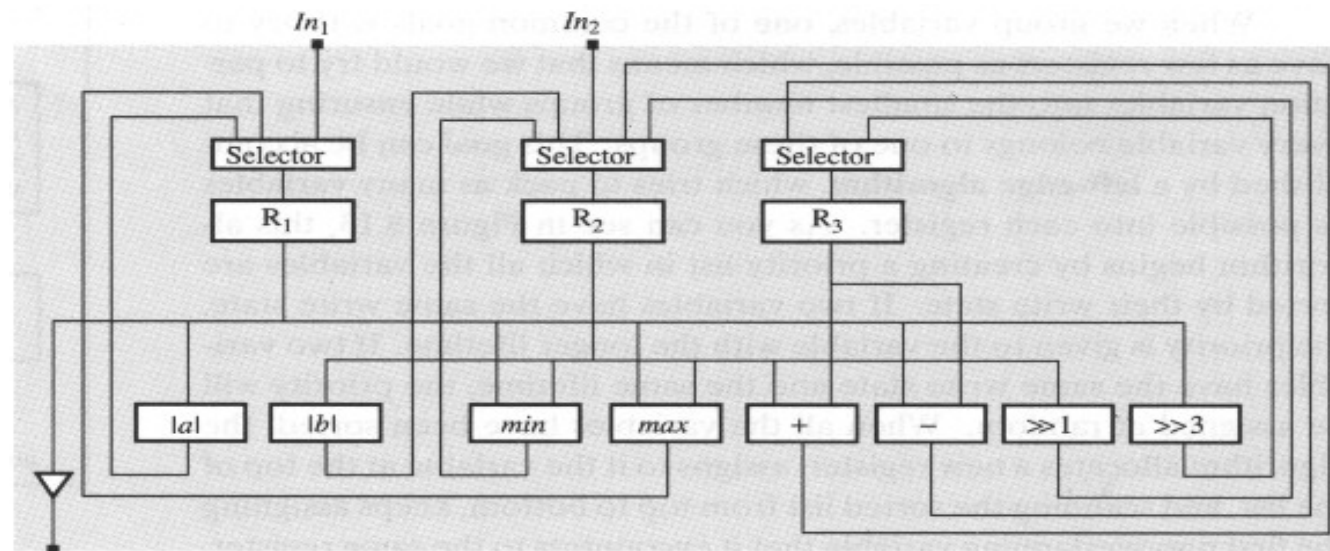
	s_0	s_1	s_2	s_3	s_4	s_5	s_6	s_7
a		x						
b		x						
t_1			x					
t_2			x					
x				x	x	x	x	
y				x				
t_4					x	x		
t_3				x				
t_5					x			
t_6							x	
t_7								x

Sorted list of variables

$R1 = \{a, t1, x, t7\}$
 $R2 = \{b, t2, y, t4, t6\}$
 $R3 = \{t3, t5\}$
 Register assignments



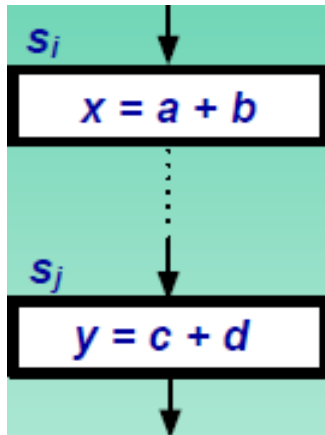
Left-edge algorithm



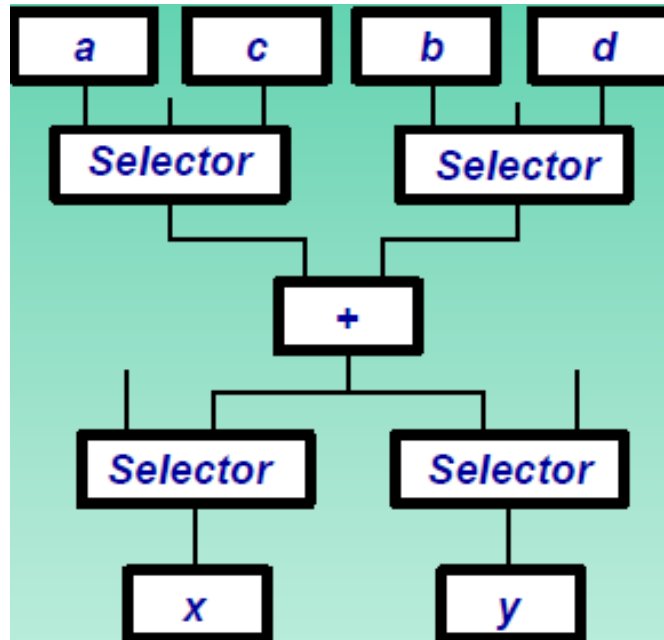
Output

Datapath schematic

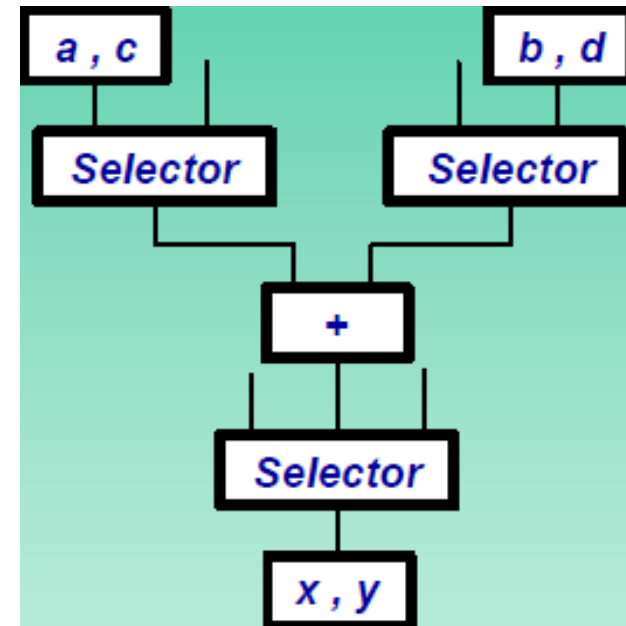
Merging variables with common Source and Destination



Partial ASM Chart

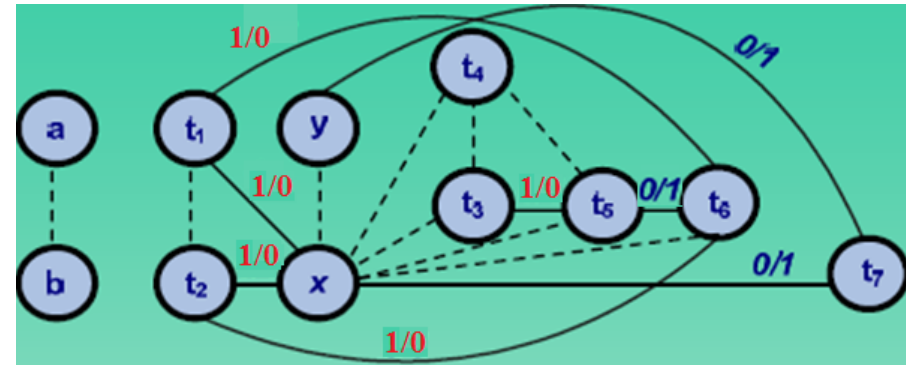


Datapath without register sharing



Datapath with register sharing

Compatibility graph for register sharing

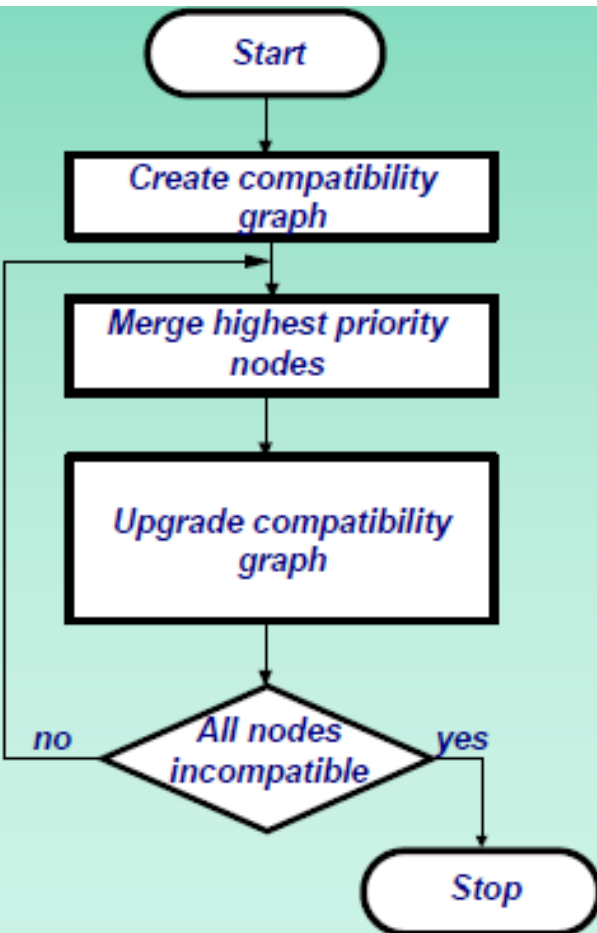


(Variables) compatibility graph

Compatibility graph: nodes and edges

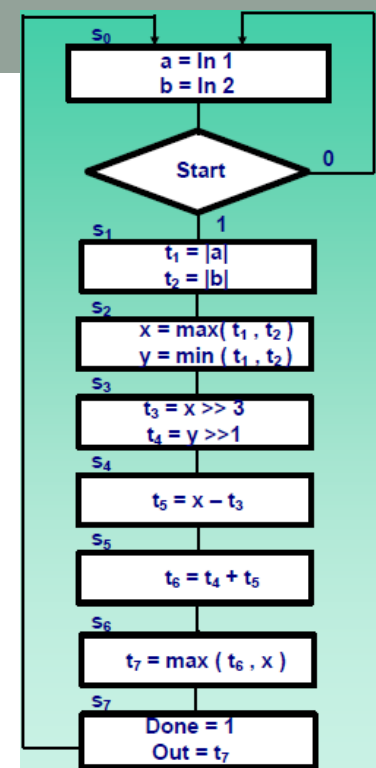
- **Node** represents a variable
- **Edge:** incompatibility edge and priority edge
 - + **Incompatibility edge** (dashed line): connect 2 nodes indicates 2 variables with overlapping lifetimes
 - + **Priority edge:** connect 2 nodes indicates 2 nonoverlapping lifetime variables that serve as the same source (input) or same destination (output) to the *same functional units or units in used library*.
 - **Priority weight (s/d) on priority edge:** the number of selector inputs can be saved
 - **s** is equal to the number of the *same functional units* that use both nodes as right operands (input).
 - **d** is equal to the number of the *same functional units* that generate results (output) for both nodes

Graph-partitioning algorithm

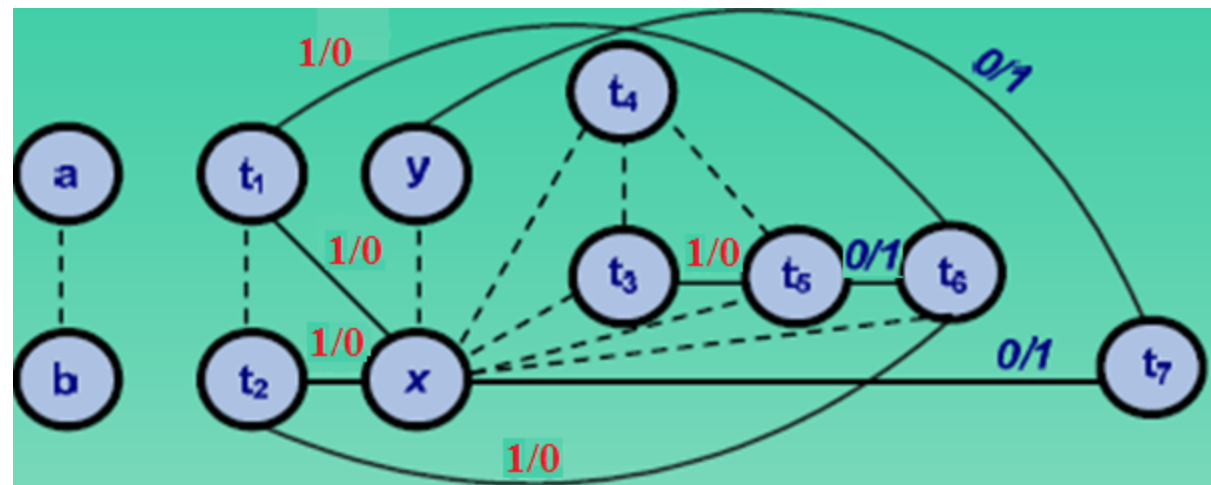


	s ₁	s ₂	s ₃	s ₄	s ₅	s ₆	s ₇
a	X						
b	X						
t ₁		X					
t ₂		X					
x			X	X	X	X	
y			X				
t ₃				X	X		
t ₄				X	X		
t ₅					X		
t ₆						X	
t ₇							X
No. of live variables	2	2	2	3	3	2	1

Variable usage (register/memory sharing)

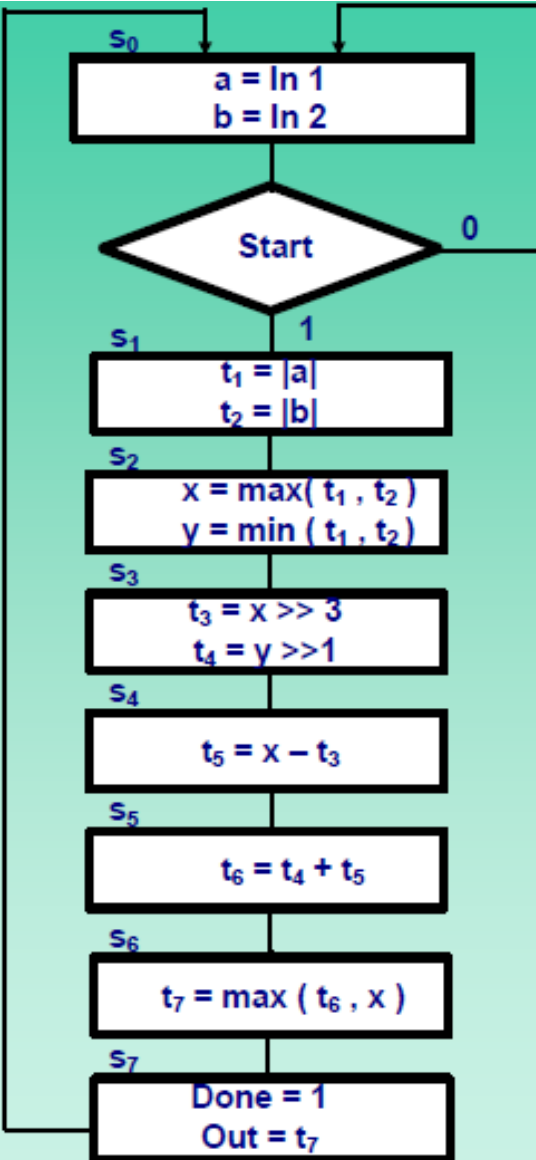


ASM Chart
(square-root approximation)

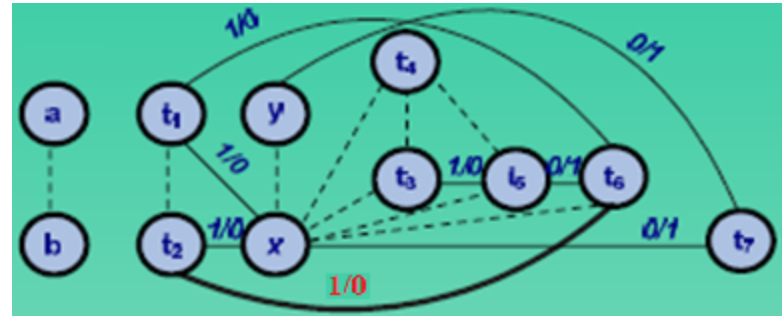


(a) Initial (variables) compatibility graph

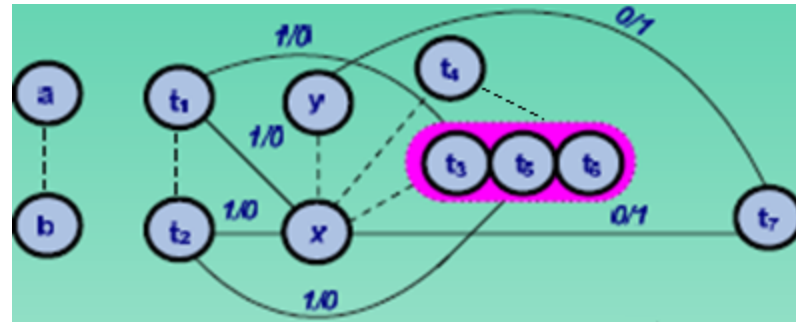
Graph-partitioning algorithm for SRA



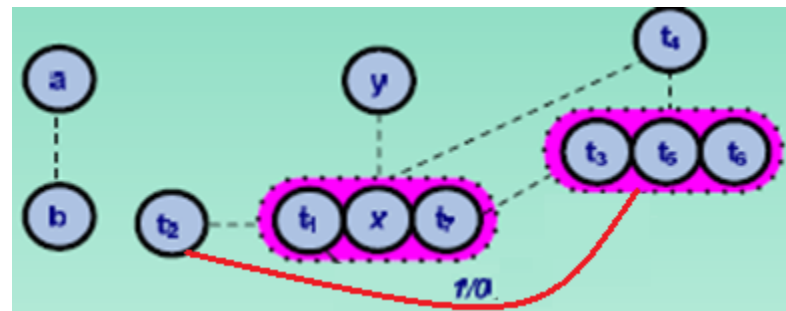
ASM Chart



(a) Initial compatibility graph

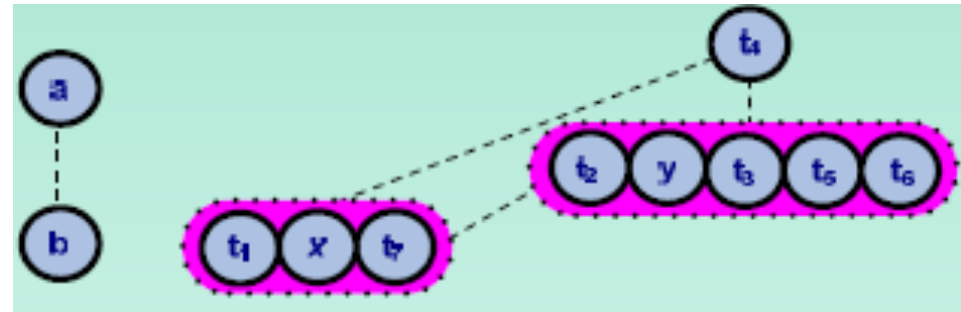
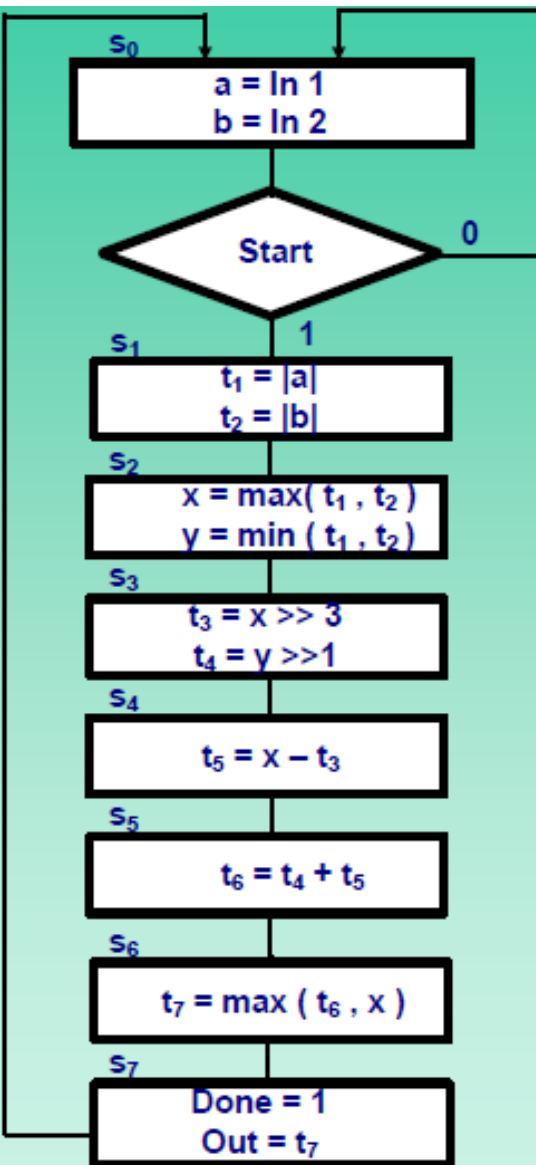


(b) Compatibility graph after merging t_3, t_5 and t_6

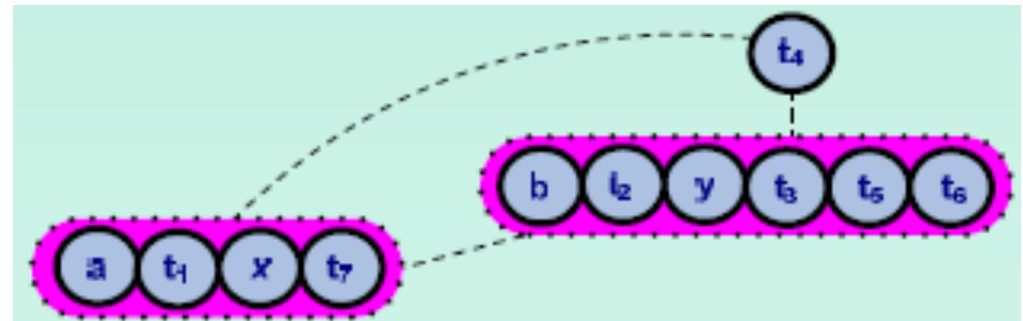


(c) Compatibility graph after merging t_1, x and t_7

Graph-partitioning algorithm for SRA



(d) Compatibility graph after merging t_2 and y



(e) Final compatibility graph

Register assignment generated by the graph-partitioning algorithm

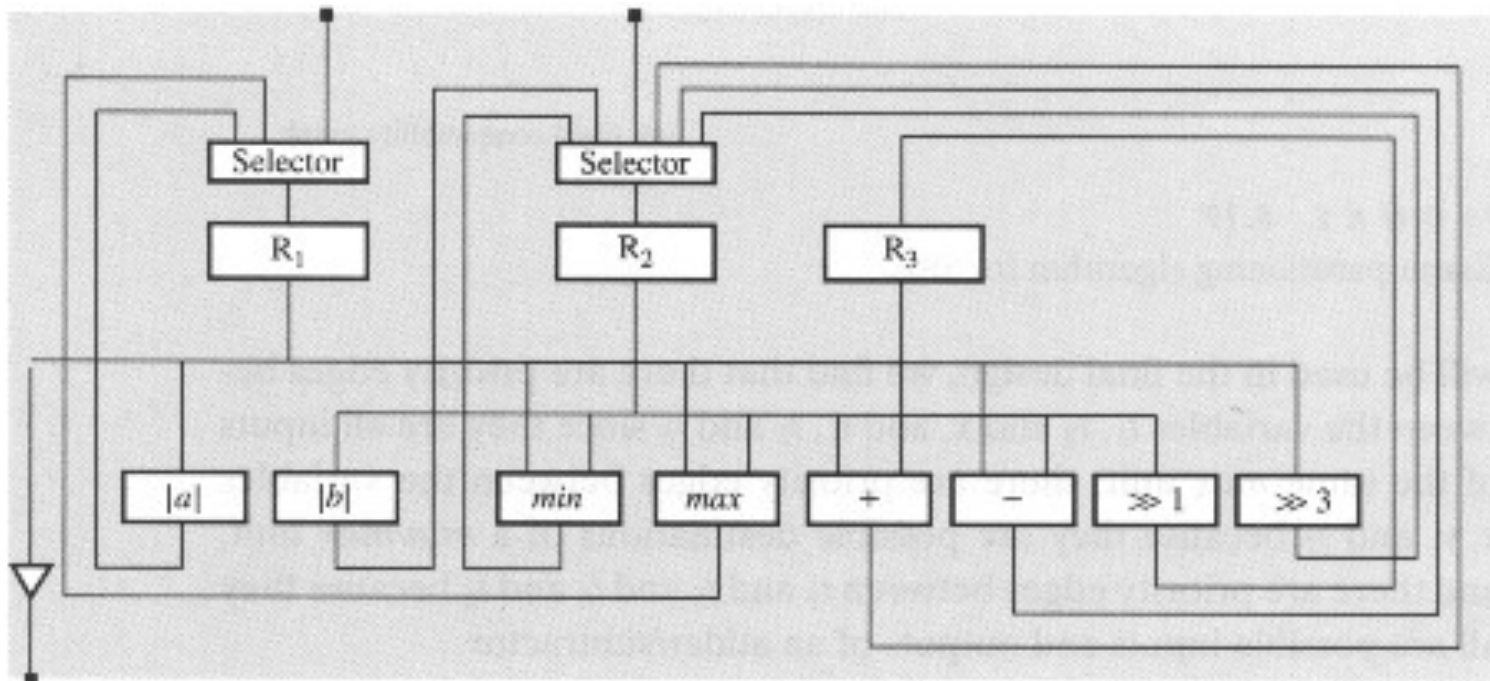
Compare *selector input numbers* with Datapath generated by the left-edge algorithm??? (slide 10)

$R1 = [a, t1, x, t7]$

$R2 = [b, t2, y, t3, t5, t6]$

$R3 = [t4]$

Register assignments

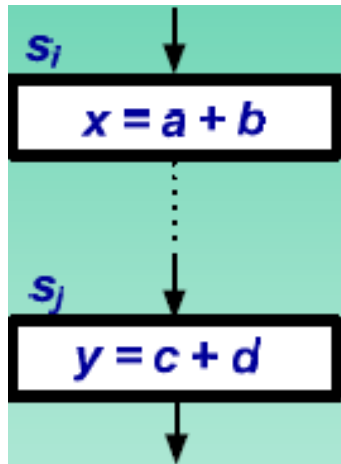


Datapath

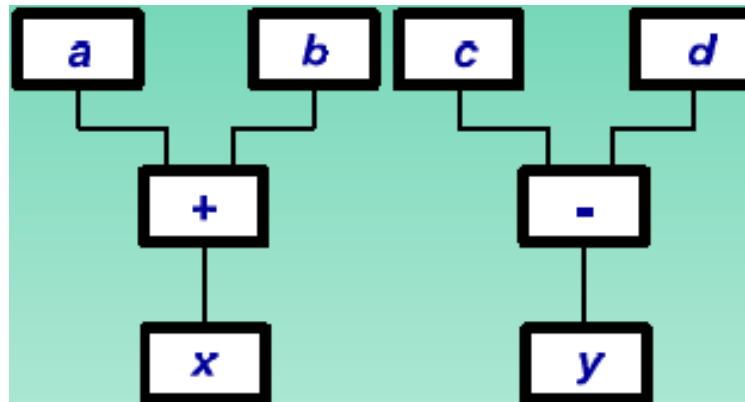
Functional-unit sharing (operator merging)

- Group non-concurrent operations
- Each group shares one functional unit
- Sharing reduces number of functional units
- Prioritized grouping by reducing connectivity
- Clustering algorithm used for grouping

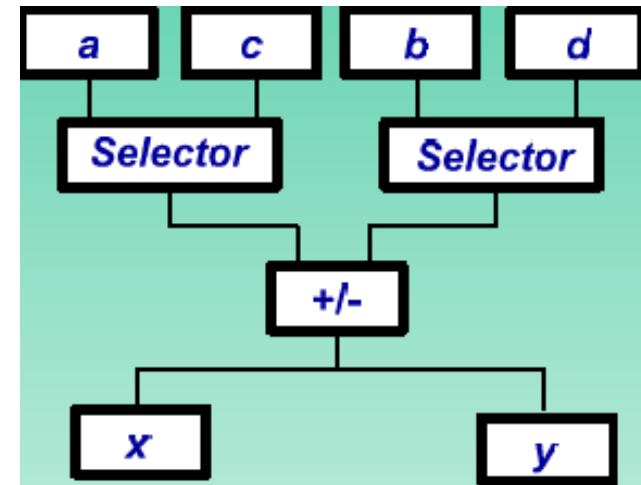
Functional-unit sharing



Partial ASM Chart

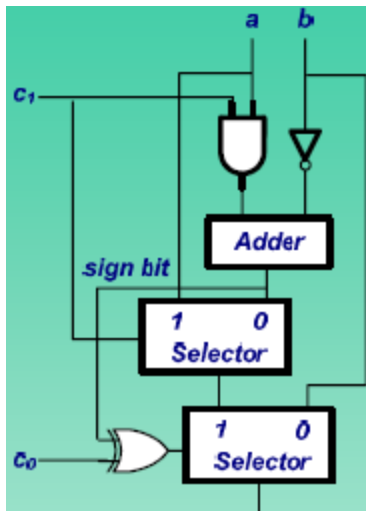


Non-shared design



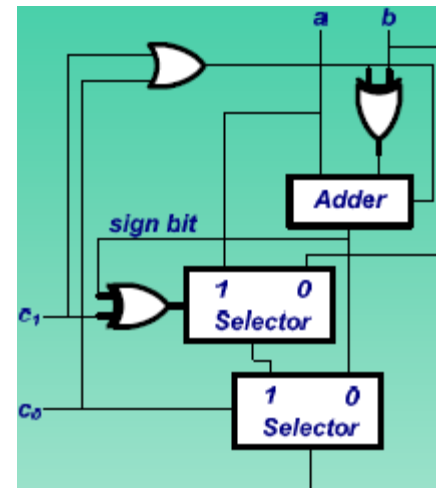
Shared design

Complex library components



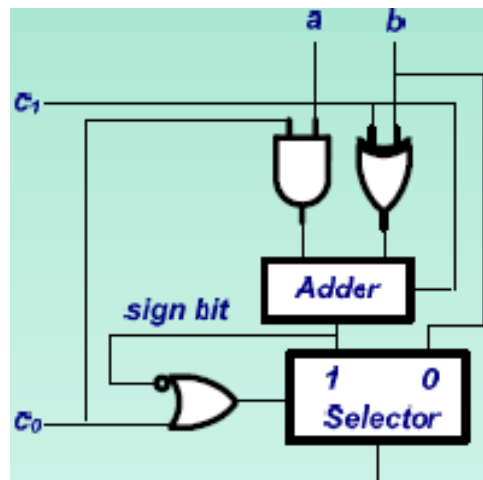
c_1	c_0	Operation
0	1	absolute
1	0	minimum
1	1	maximum

Unit for computing minimum, maximum and absolute value



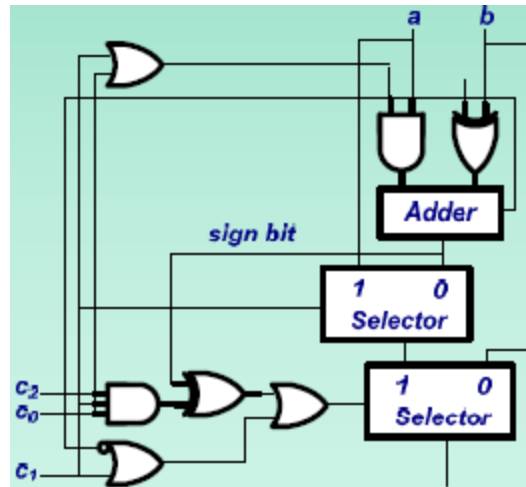
c_1	c_0	Operation
0	0	addition
0	1	minimum
1	0	subtraction
1	1	maximum

Unit for computing addition, subtraction, minimum and maximum



c_1	c_0	Operation
1	0	addition
0	1	absolute
1	1	subtraction

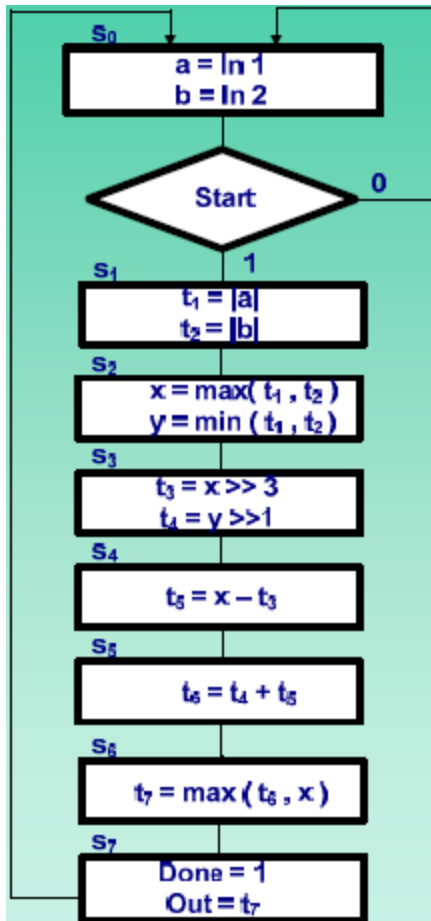
Unit for computing addition, subtraction, and absolute value



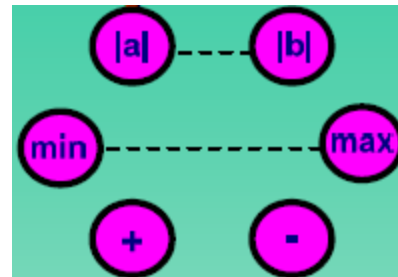
c_2	c_1	c_0	Operation
0	0	1	addition
1	0	0	absolute
1	0	1	subtraction
1	1	0	minimum
1	1	1	maximum

Unit for computing addition, subtraction, minimum, maximum and absolute value

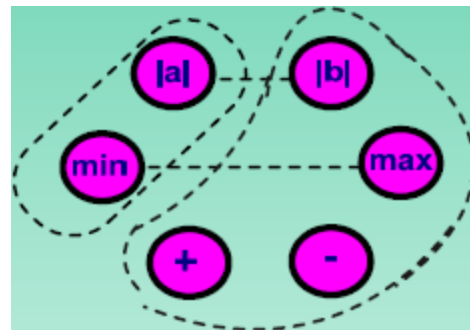
Operator merging for SRA implementation



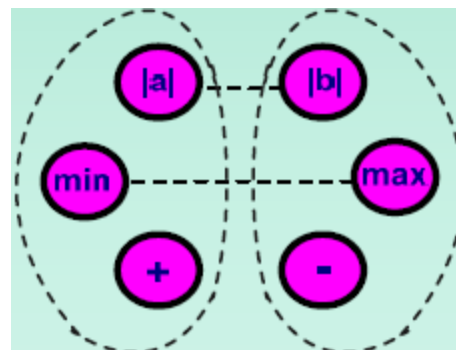
ASM Chart



(a) Compatibility graph
(single function unit)



(c) Merging alternative
(simple merging operator)



(e) Merging alternative
(simple merging operator)

Component Unit	AND Logic	Invert Logic	EX-OR Logic	Adder	Selector
a		1		1	1
b		1		1	1
min		1		1	1
max		1		1	1
+				1	
-		1		1	
Total		5		6	4

(b) Cost table graph (a)

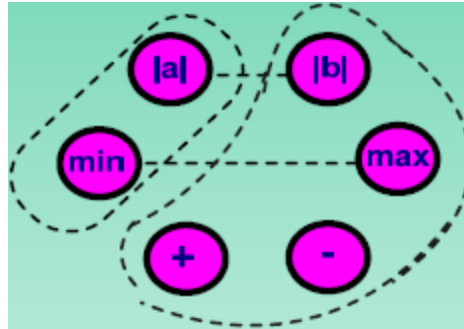
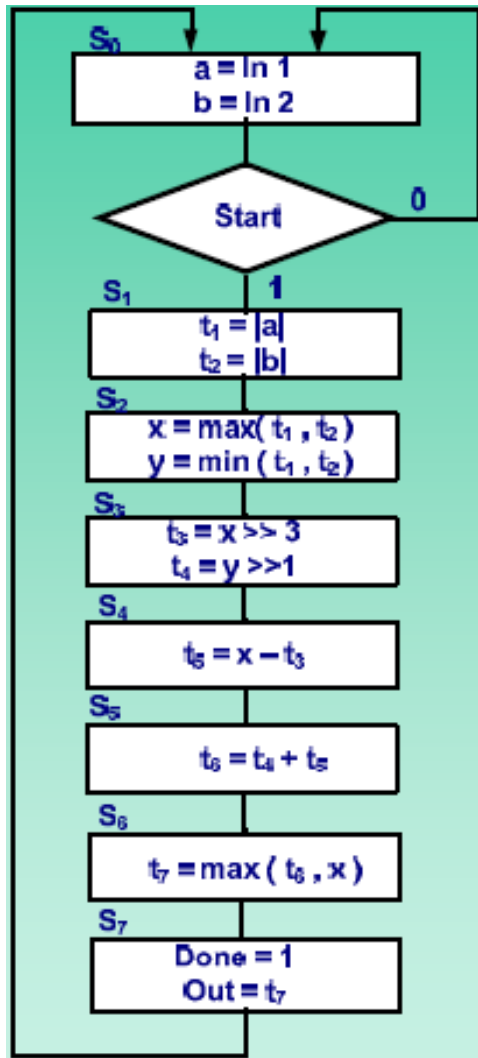
Component Unit	AND Logic	Invert Logic	EX-OR Logic	Adder	Selector
[a / min]	1	1		1	2
[b / max / + / -]	1		1	1	2
Total	2	1	1	2	4

(d) Cost table graph (c)

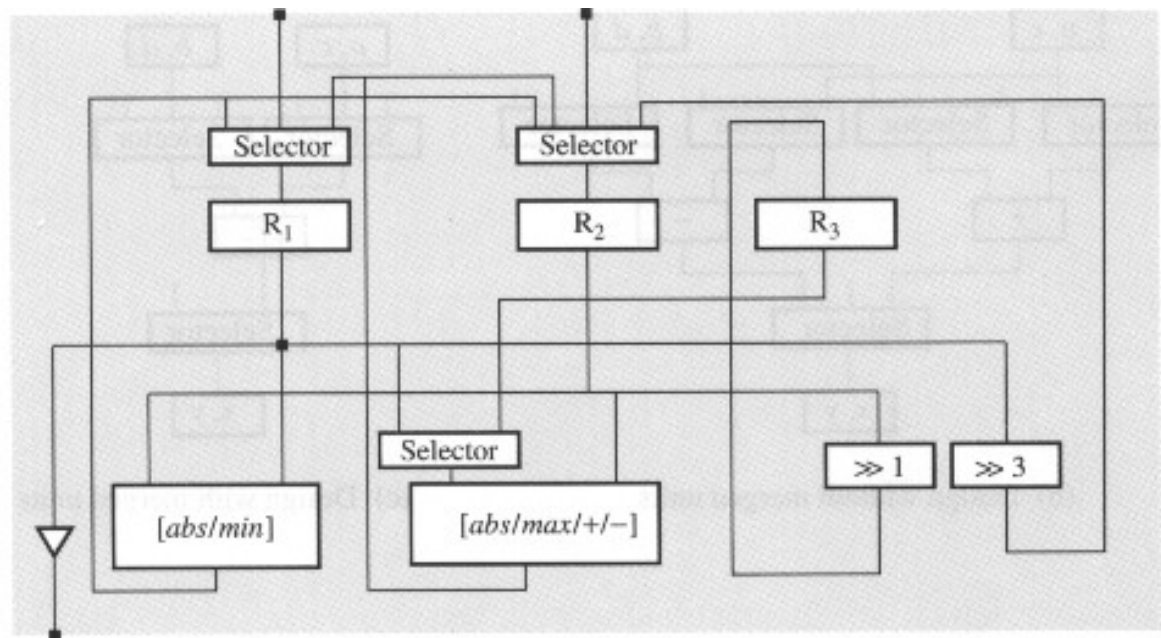
Component Unit	AND Logic	Invert Logic	EX-OR Logic	Adder	Selector
[a / min / +]	1		1	1	2
[b / max / +]	1	1		1	2
Total	2	1	1	2	4

(f) Cost table graph (e)

Datapath connectivity



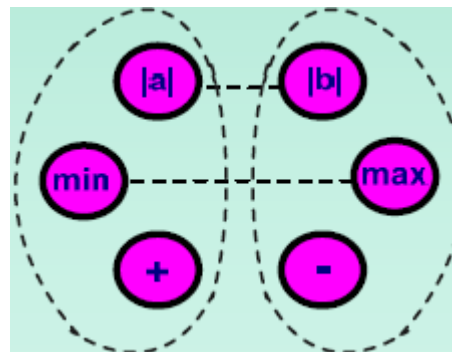
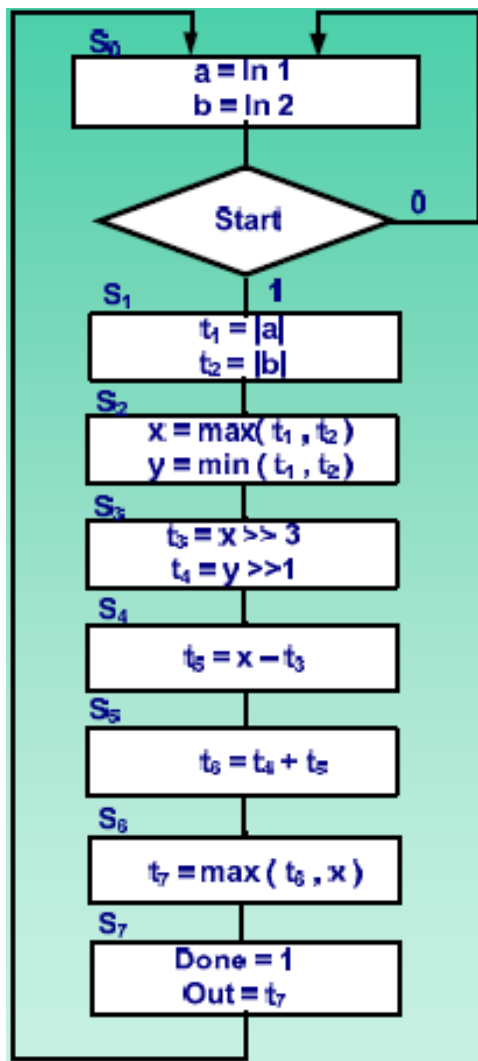
Compare *selector input numbers* with Datapath in slide 17 ???



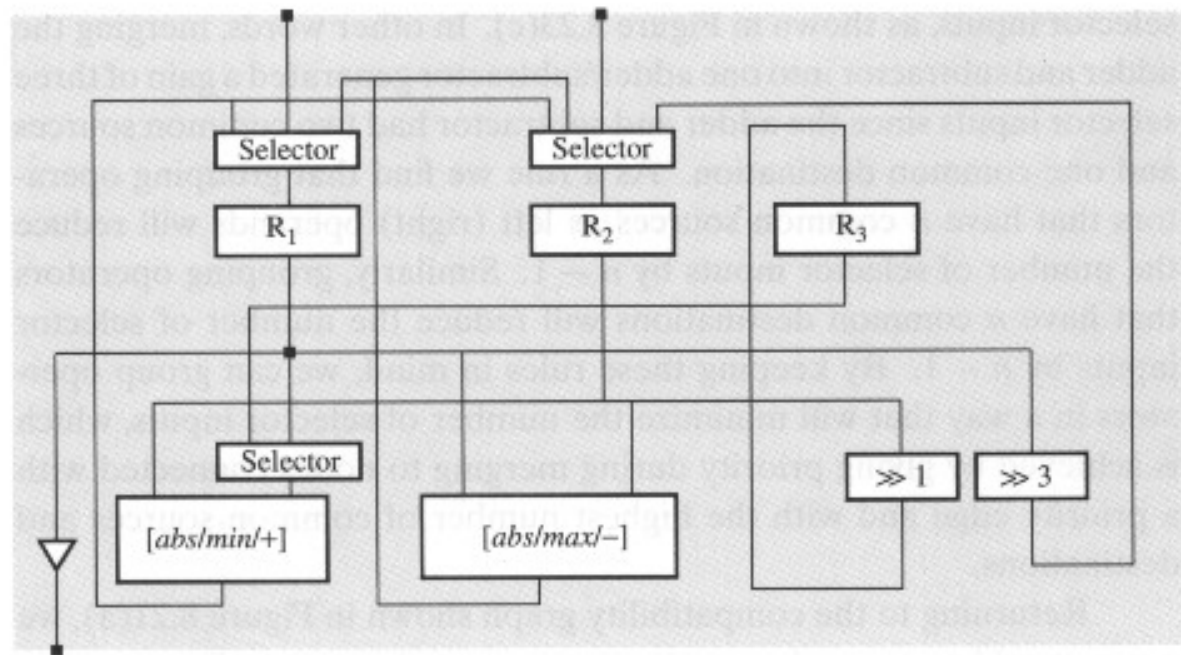
ASM Chart

Datapath schematic

Datapath connectivity

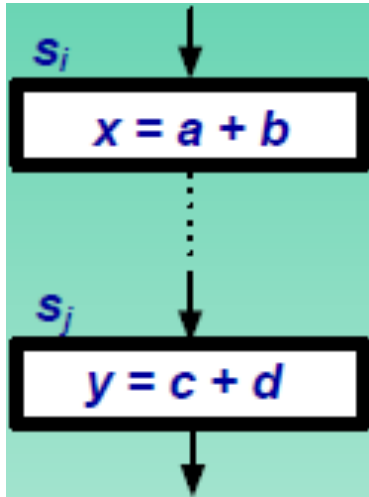


Compare *selector input numbers* with Datapath in slide 17 ???

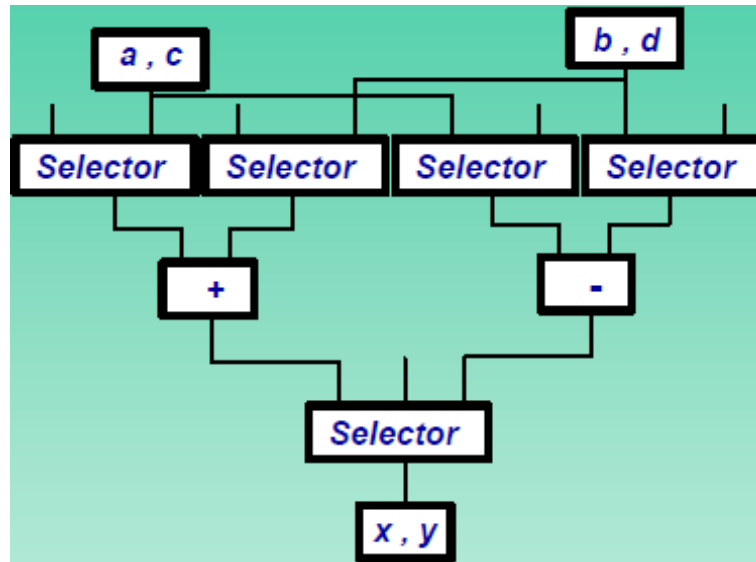


Datapath schematic

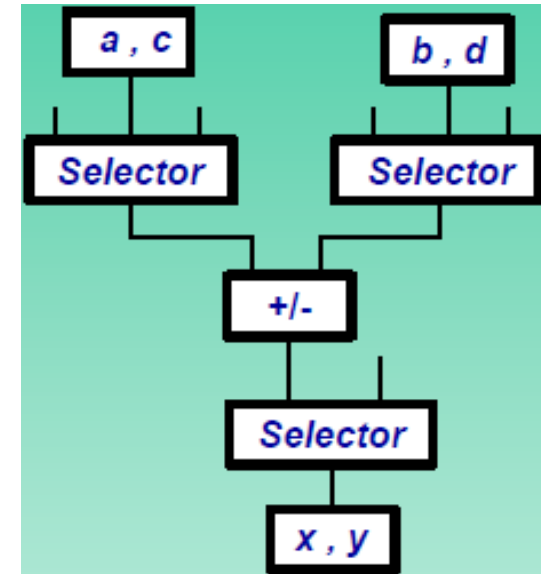
Priorities in unit merging



(a) Partial ASM Chart



(b) Design without merged units

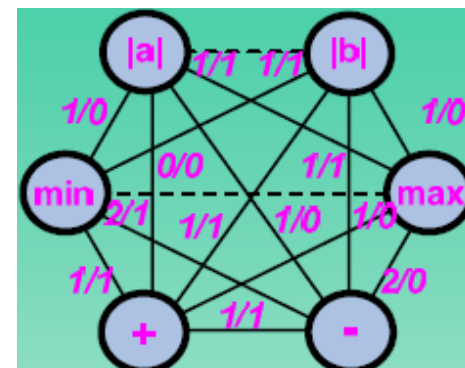


(c) Design with merged units

Compatibility graph for functional-unit sharing

Compatibility graph: nodes and edges

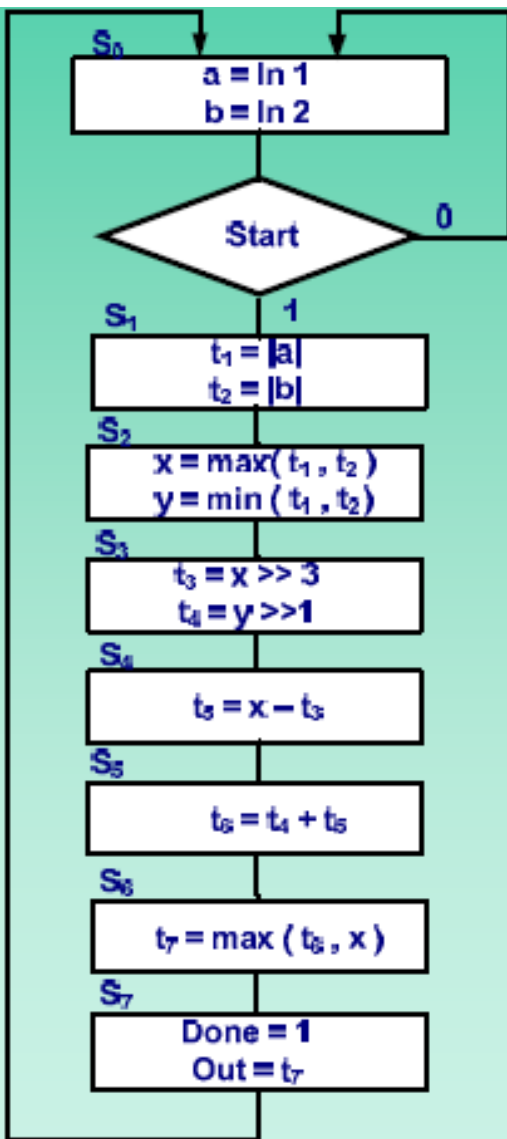
- **Node** represents a operator
- **Edge:** incompatibility edge and priority edge
 - + **Incompatibility edge** (dashed line): connect 2 nodes indicates 2 operators that are occurred concurrently in the same state.
 - + **Priority edge:** connect 2 nodes indicates 2 operators that have the *operator's similarity in construction* (depend on component libraries) **or** the *substantially reduce the cost* (depend on priority weight) of the datapath's connection.
 - **Priority weight (s/d) on priority edge:** the number of connections can be saved
 - **s** is equal to the number of *common sources* between two operators
 - **d** is equal to the number of *common destinations* between two operators.



(Operators) compatibility graph

Note: If the ASM chart has more than one operator (e.g. **max**) in ASM chart, the *maximum value* of **s or d** from each one to other operator will be chosen.

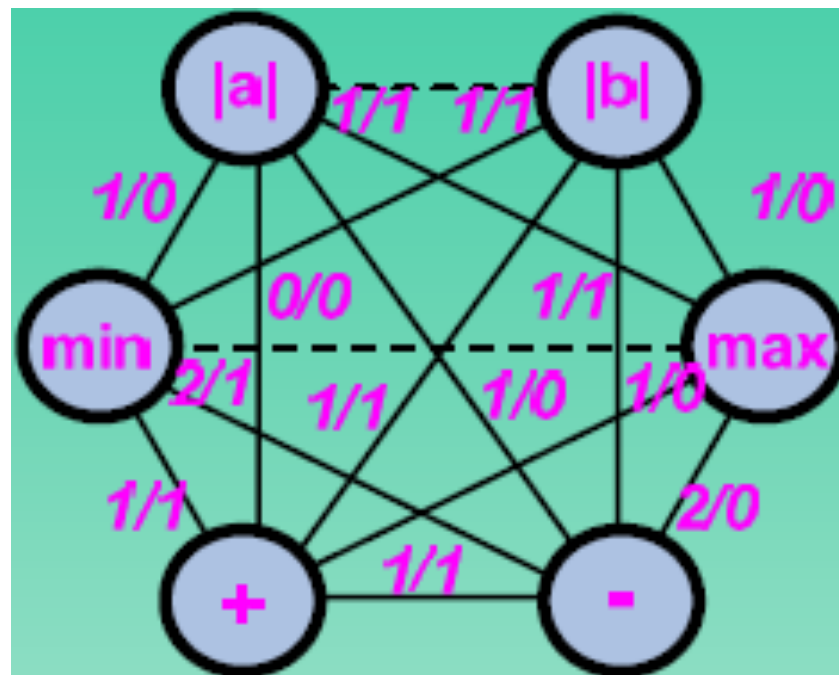
Unit merging for SRA datapath



ASM Chart

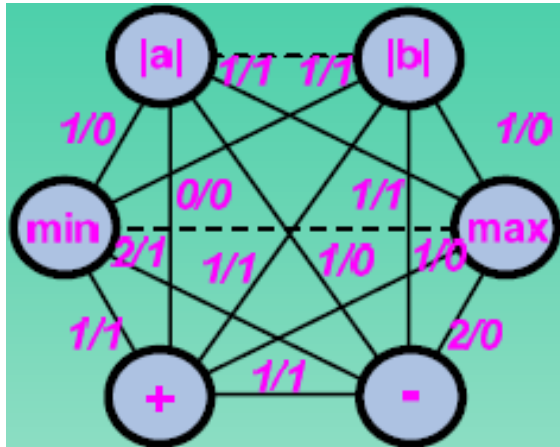
$R1 = [a, t1, x, t7]$
 $R2 = [b, t2, y, t3, t5, t6]$
 $R3 = [t4]$

Register assignment in register sharing with graph-partitioning algorithm (slide16)

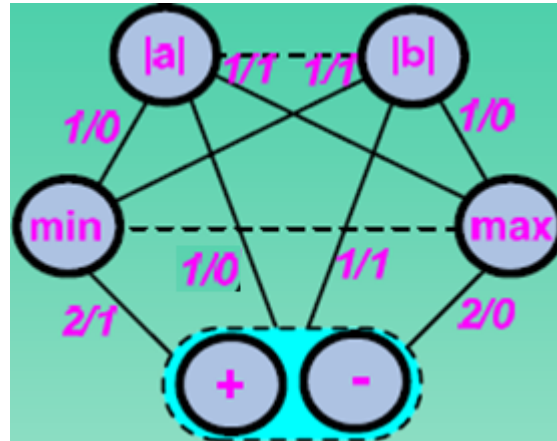


(Operators) Compatibility graph

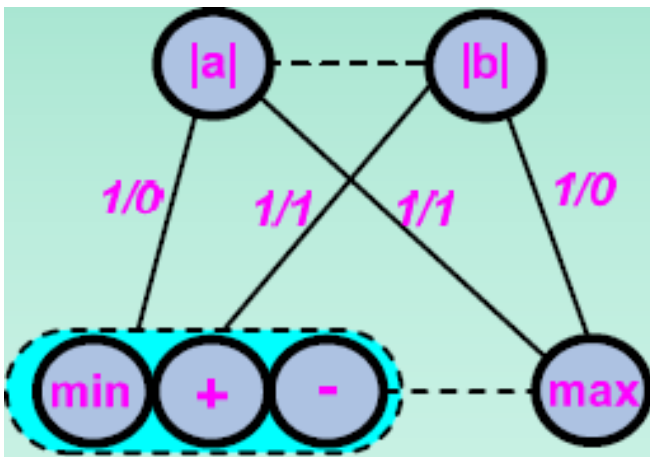
Unit merging for SRA datapath



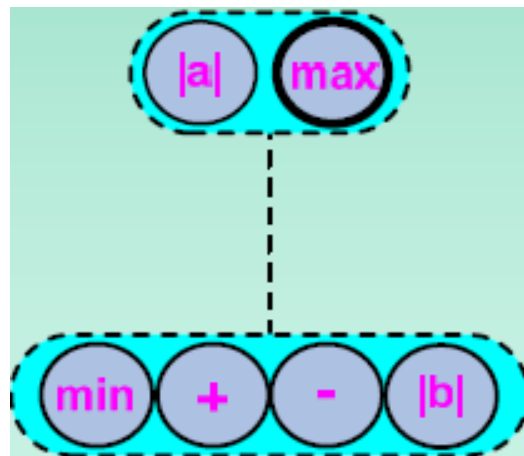
(a) Compatibility graph



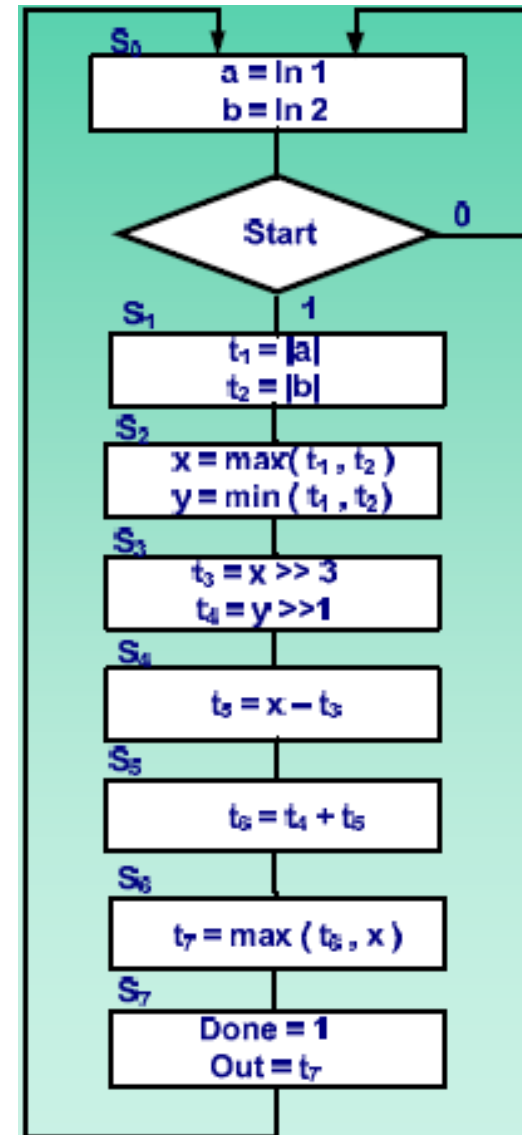
(b) Compatibility graph
after merging of $+$ and $-$
(operator's similarity grouping)



(c) Compatibility graph
after merging of $\min$, $+$ and $-$
(datapath's connection grouping)



(d) Final graph partitions
(datapath's connection grouping)



ASM Chart

SRA datapath generated by prioritized partitioning (use register and functional-unit sharing)

$$R_1 = [a, t_1, x, t_7]$$

$$R_2 = [b, t_2, y, t_3, t_5, t_6]$$

$$R_3 = [t_4]$$

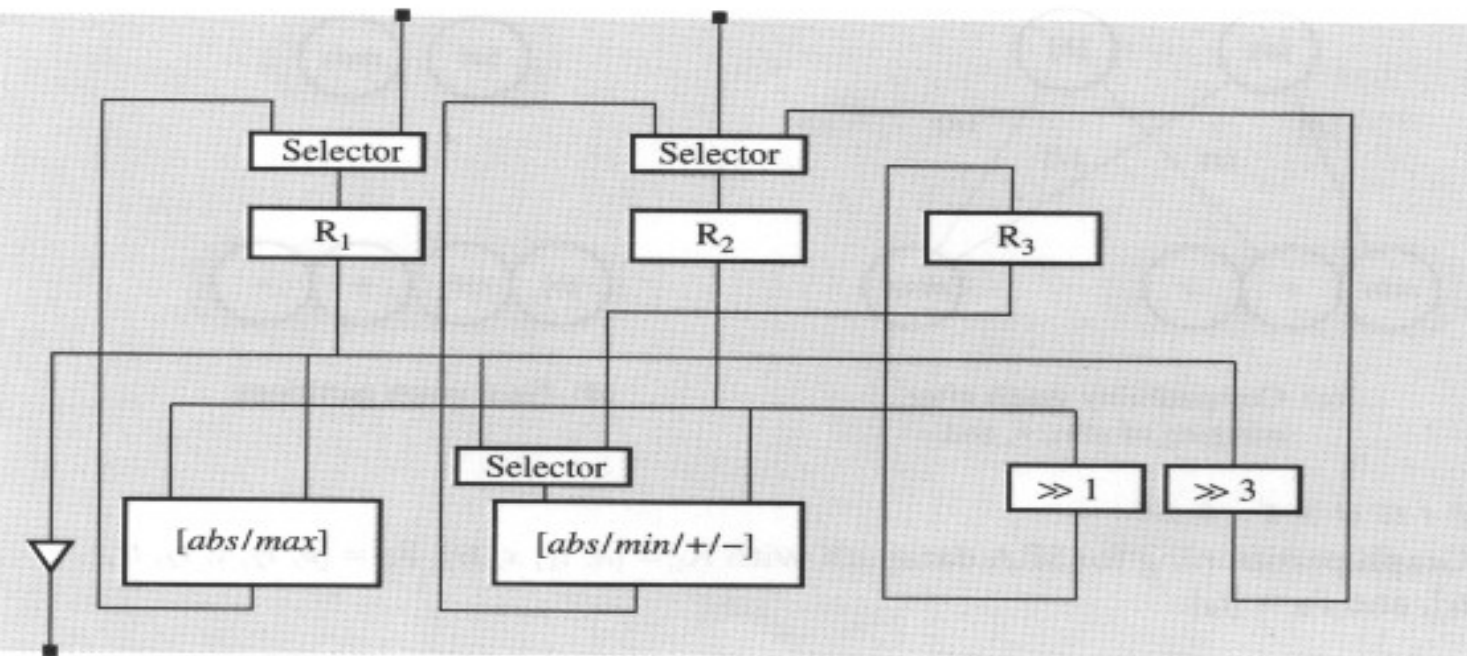
$$AU_1 = [|b| / \min / + / -]$$

$$AU_2 = [|a| / \max]$$

$$SH_1 = [>>1]$$

$$SH_2 = [>>3]$$

(a) Register and functional unit allocation



(b) Datapath schematic

Compare selector
input numbers with
Datapath in slide
16,21,22 ???

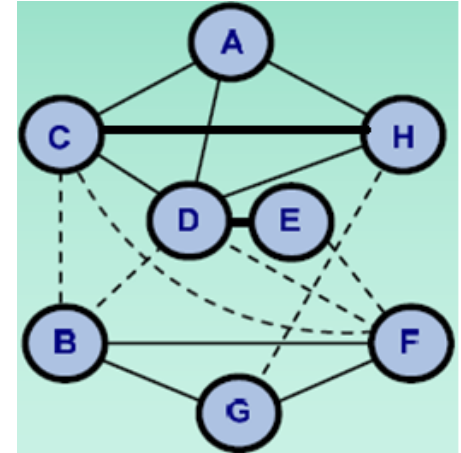
Bus sharing (connection merging)

- **Group connections that are not used concurrently**
- **Each group forms a bus**
- **Connection merging reduces number of wires**
- **Clustering algorithm is demonstrated**

Compatibility graph for bus sharing

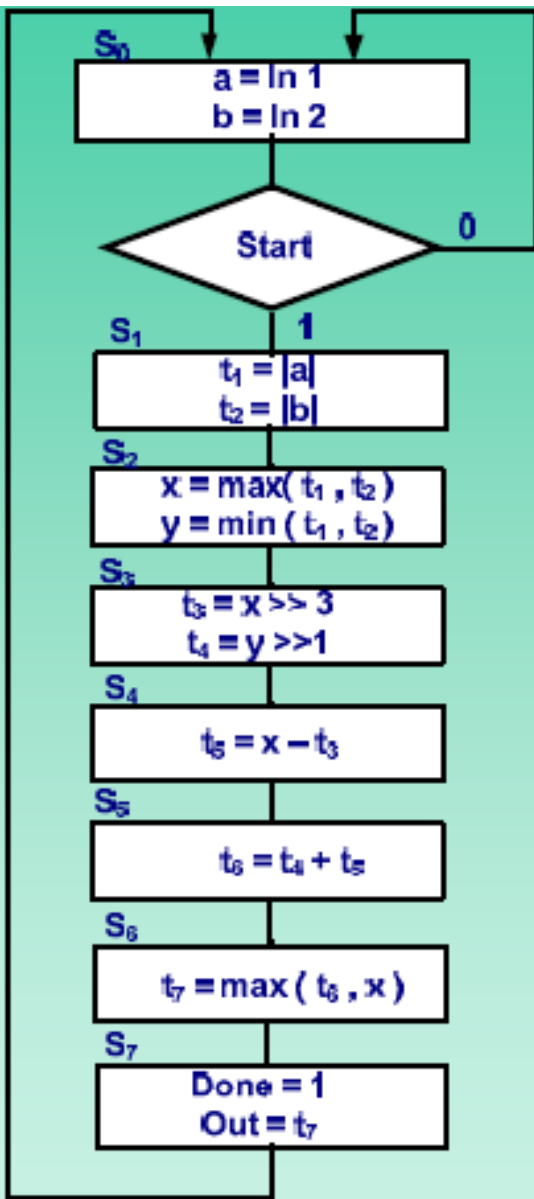
Compatibility graph: nodes and edges

- **Node** represents each connection
- **Edge:** incompatibility edge and priority edge
 - + **Incompatibility edge** (dashed line): connect 2 nodes indicates 2 connections that *do not originate from* the same source but are used at the same time.
 - + **Priority edge:** connect 2 nodes indicates 2 connections that have a common source or a common destination.
 - **Priority weight (s/d) on priority edge:** **No** → using a graph-partitioning algorithm to group connections in a way that will maximize the number of priority edges included in all groups.

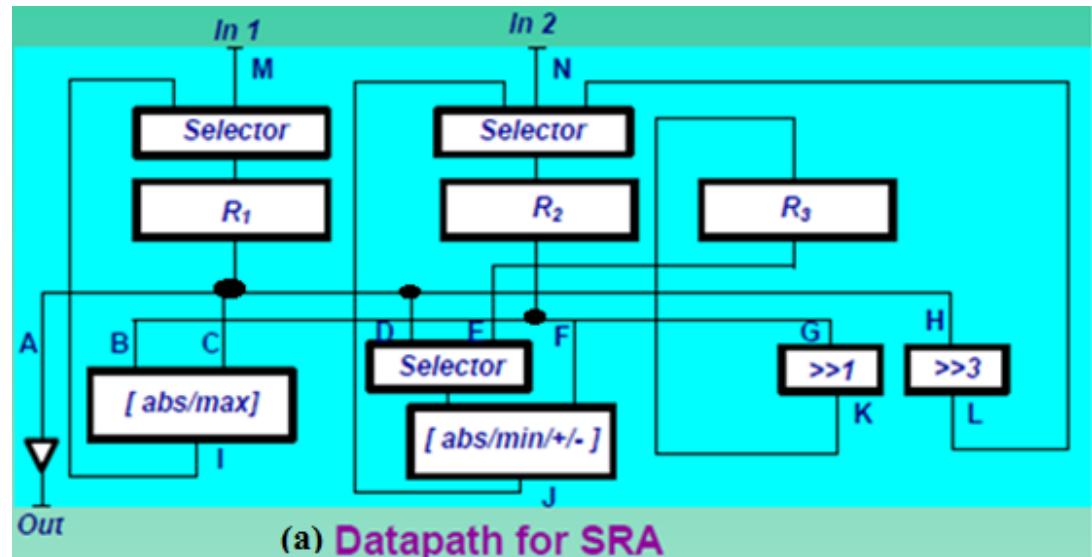


(connection) compatibility graph

Connection merging in SRA datapath



ASM Chart



(a) Datapath for SRA

$R1 = [a, t1, x, t7]$

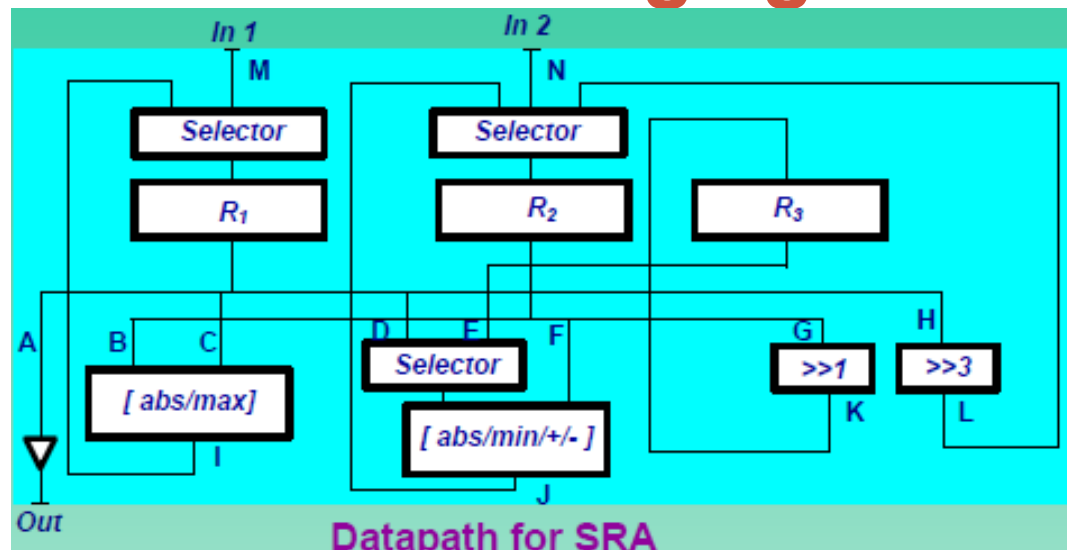
$R2 = [b, t2, y, t3, t5, t6]$

$R3 = [t4]$

	S ₀	S ₁	S ₂	S ₃	S ₄	S ₅	S ₆	S ₇
A								X
B		X	X				X	
C			X				X	
D			X		X			
E						X		
F		X	X		X	X		
G				X				
H			X	X				
I		X	X				X	
J		X	X		X	X		
K				X				
L				X				
M	X							
N	X							

(b) Connectivity usage table

Connection merging in SRA datapath



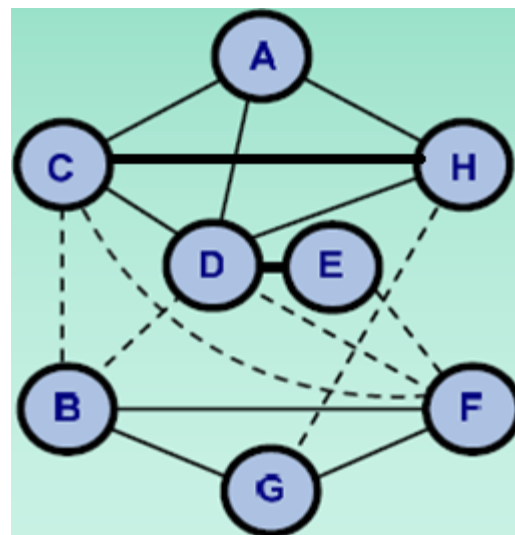
$R1 = [A, C, D, H]$

$R2 = [B, F, G]$

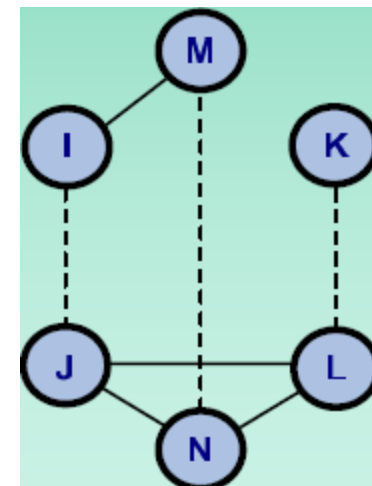
$R3 = [E]$

	s ₀	s ₁	s ₂	s ₃	s ₄	s ₅	s ₆	s ₇
A								X
B							X	
C		X	X				X	
D			X		X			
E						X		
F		X	X		X	X		
G				X				
H				X				
I		X	X				X	
J		X	X		X	X		
K				X				
L				X				
M	X							
N	X							

(b) Connectivity usage table

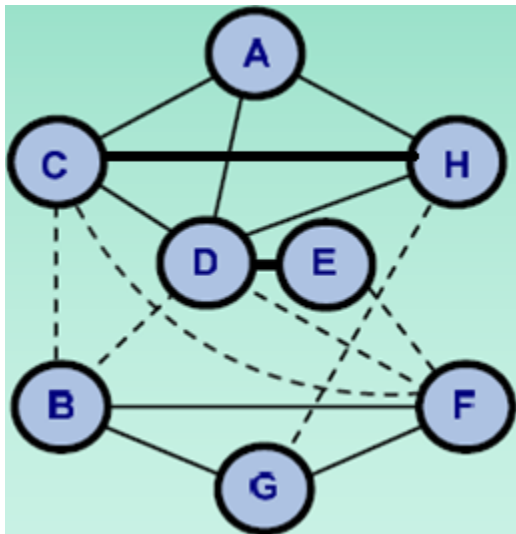


(c) Compatibility graph for input buses (AU and SH inputs)

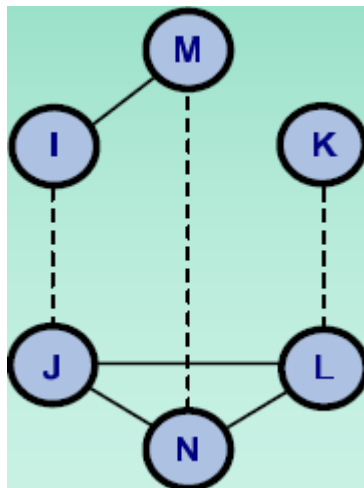


(d) Compatibility graph for output buses (AU and SH outputs)

Connection merging in SRA datapath



(c) Compatibility graph for input buses



(d) Compatibility graph for output buses

Bus1 = [A, C, D, E, H]

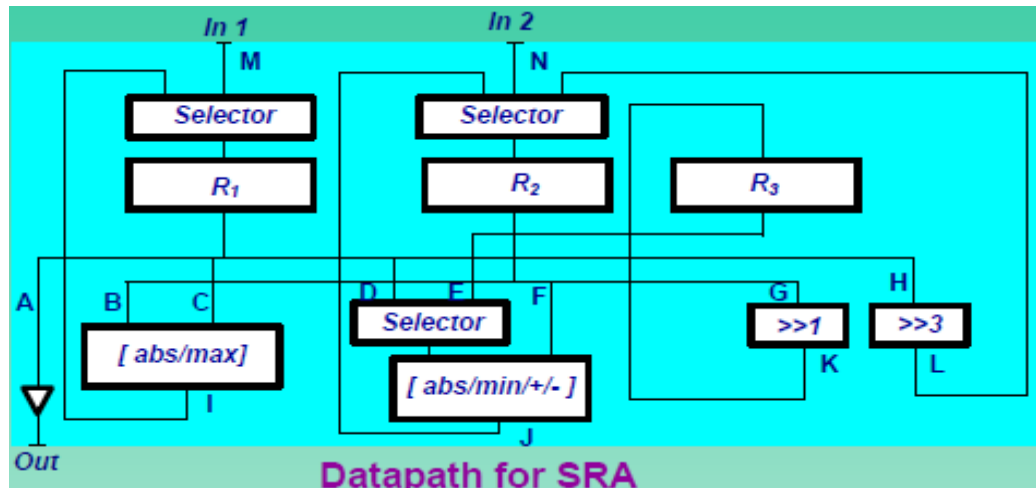
Bus2 = [B, F, G]

Bus3 = [I, K, M]

Bus4 = [J, L, N]

(e) Bus assignment

Connection merging in SRA datapath



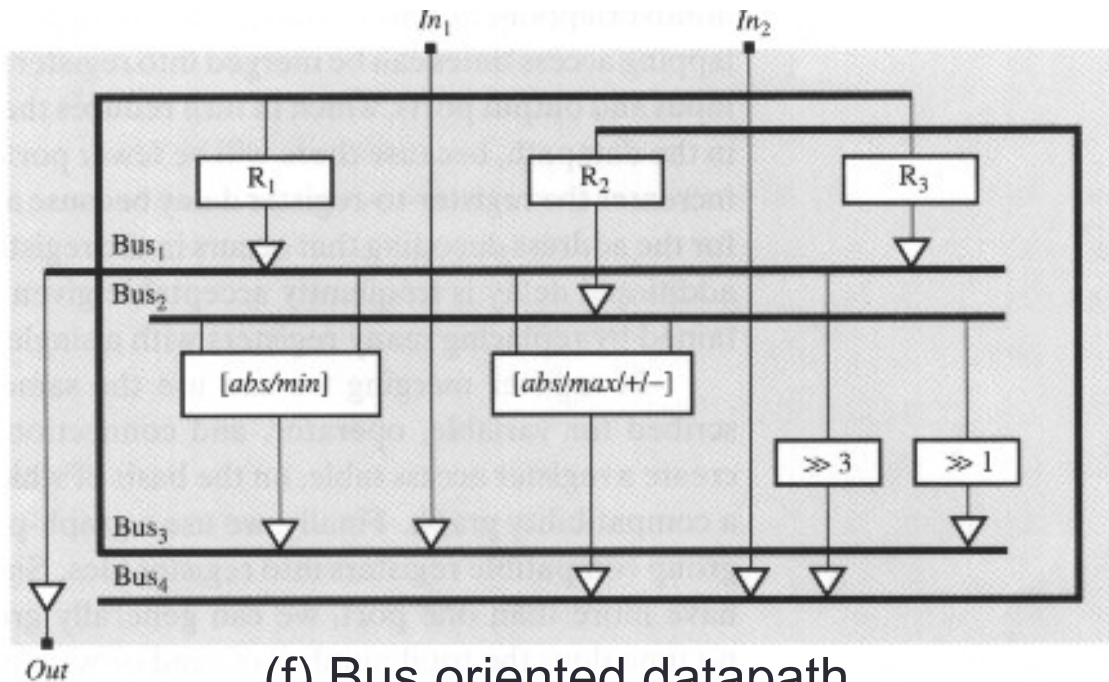
Bus1 = [A, C, D, E, H]

Bus2 = [B, F, G]

Bus3 = [I, K, M]

Bus4 = [J, L, N]

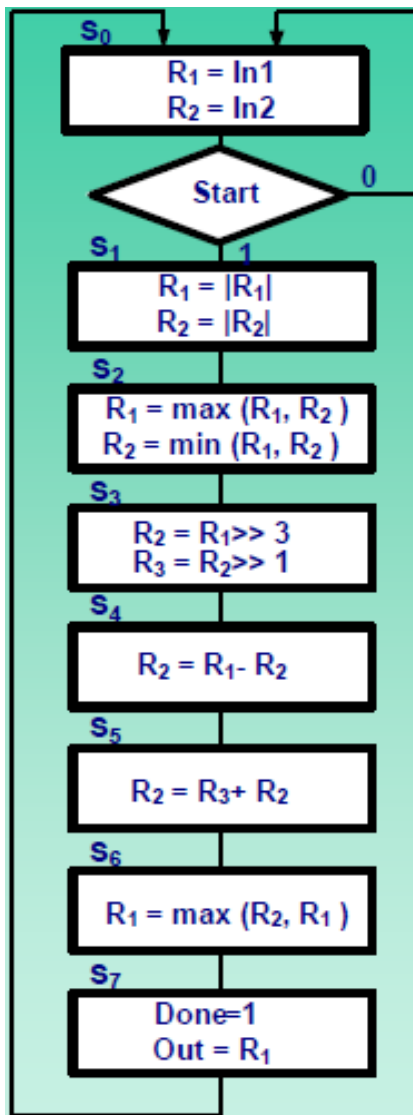
Bus assignment



Register merging

- **Group register with nonoverlapping accesses**
- **Each group assigned to one register file**
- **Register grouping reduces number of ports, and therefore number of buses**
- **Demonstration with clustering algorithm**

Register merging



ASM Chart

$R1 = [a, t1, x, t7]$

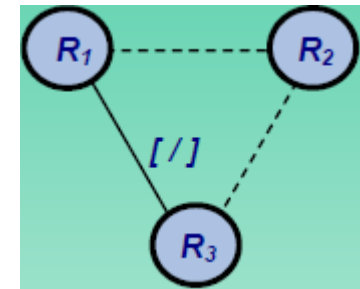
$R2 = [b, t2, y, t3, t5, t6]$

$R3 = [t4]$

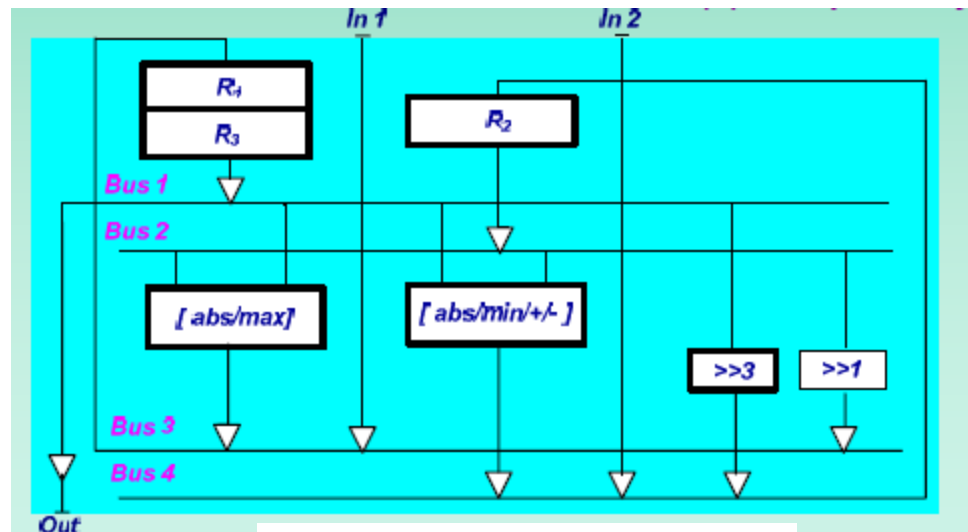
(a) Register assignment

	s ₀	s ₁	s ₂	s ₃	s ₄	s ₅	s ₆	s ₇
R ₁	▶▶▶▶▶▶▶▶▶▶							
R ₂	▶▶▶▶▶▶▶▶▶▶							
R ₃				▶▶▶▶▶▶▶▶▶▶				

(b) Register access table



(c) Compatibility graph



(d) Datapath schematic